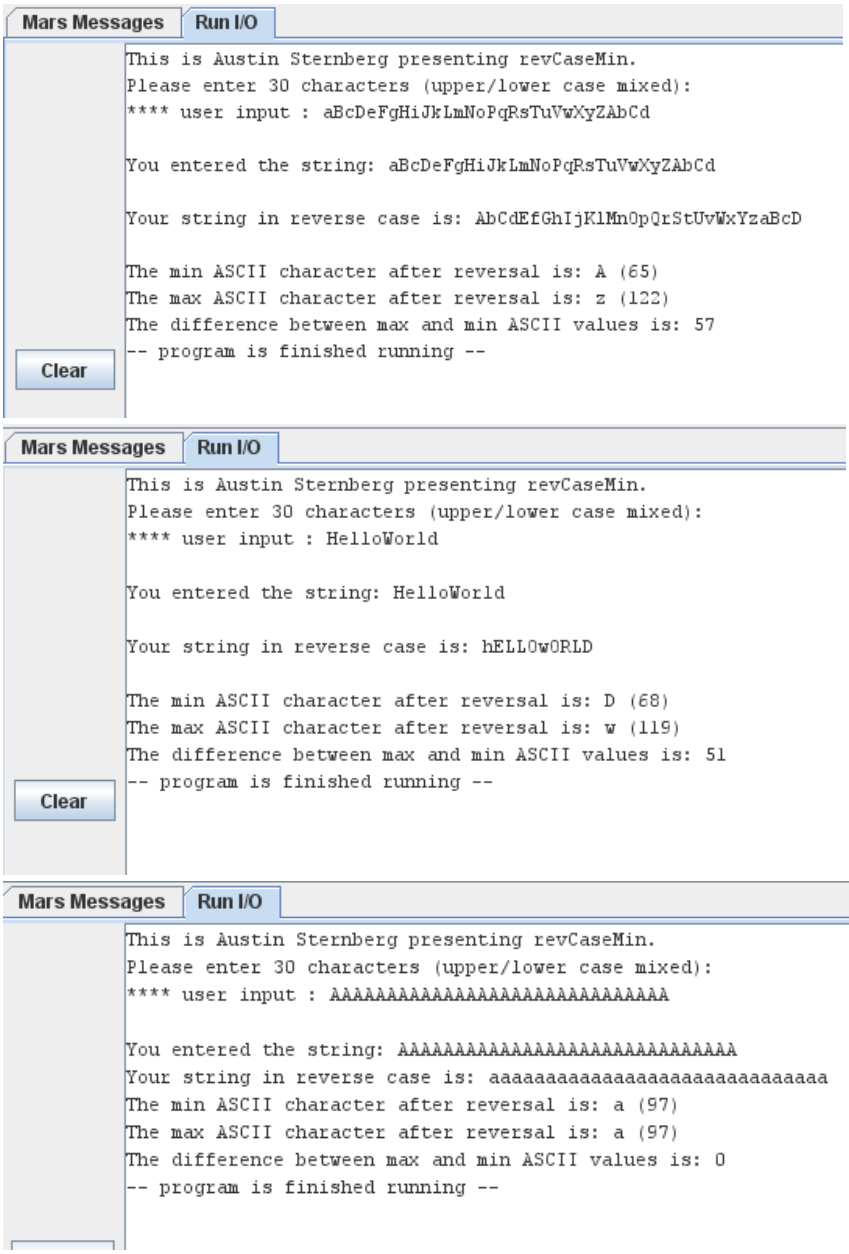
Project 5 CO Report - revCaseMin

1. Austin Sternberg
2. revCaseMin.asm
3. This program is split into four different parts, the main processes, and the two functions required to calculate the maximum and minimum ascii values, and the revCase function that inverts the case of the string input. The process starts by asking the user to input a 30-character string of upper- and lower-case letters, (lines 64-84). Lines 86-91 quickly print the string out for confirmation, before going to lines 92-100, where the boilerplate is printed out and the process goes to the revCase function. This function (lines 174-237) starts by setting up a loop counter and stack memory needed, then loops through the while and loop counter sections, inverting the case of the letter as needed. Lines 202-208 are the system in which the conversions are made, either adding or subtracting values to convert the case type. After the loop is exited, the last section of the function prints out the reversed string and hops over to the findMin function. This function (lines 241-283) also initializes a loop variable, then loops through the string. If any of the characters have a lesser value than the current value stored in $t9, such value will be stored in the register instead. Once the function has checked each character, it will break out of the loop, store the minimum value to the stack, and jump back to the very end of the revCase function. The end of revcase will load the minimum value back into the registers, then dealocate the stack, and jump back to the main function. The next section of the main program (lines 102-130) will print out the minimum value to console along with the boilerplate, so the user understands the output. Lines 132-157, invoke the findMax function and then print out the maximum value. Like the findMin function, the findMax function (lines 285-318) will loop through every character in the string, replacing the stored value if it finds a larger one down the string. Lastly, lines 158-172 will calculate the difference between the minimum and maximum values, print them out to console, and tthenexit the program.

4. 

5. This was definitely the most challenging project of the semester. Learning how to get assembly programs to invoke functions, jump out of main processing, run the function, and then return to main processing was quite a step up from the last projects. Learning how the jump and link command (jal) worked was the foundation of figuring out how to make such a system work in MIPS. The interesting part in how the jal instruction works is that it can make multiple jumps before returning to the outright original invocation.